[Title page]

[0. Abstract]

Abstract

[WRITE LAST]

Application Security Strategy

Software Development Lifecycle

When designing and developing an application it is vital that the application be developed and have security planned for at the beginning. The Software Development Lifecycle (SDLC) is used to help facilitate and provide a path to develop applications. As shown in Figure 1, the SDLC starts off with the Planning and Analysis phase which is the most vital and pivotal phase of the SDLC, as shown in Figure 1. In this phase, the supervisors of the Exxon Software Security Team will work with the Exxon Board of Directors to plan the development of the application by making responsibility charts, managing a Secure Software Development Lifecycle (SSDLC), and by making policies for the developers to ensure security during development. The second most important phase of the SDLC is the define and design phase, where the development team generates and curates a list of functional requirements, security requirements, metrics, and a Software Requirement Specification (SRS). Regarding the SDLC, functional requirements are specifiable requirements that detail what the program is supposed to do to complete its primary function, as well as any needs or requests from stakeholders. Contrarily,

security requirements define what the program should not be able to do; to restrict the program's function to not expose common risks and vulnerabilities.

Figure 1

The Software Development Lifecycle

Note. This figure shows the broad phases of the Open Web Application Security Project (OWASP) SDLC representation. From "OWASP in SDLC", by owasp.org, (2022). (https://owasp.org/www-project-integration-standards/writeups/owasp_in_sdlc/)

In order to ensure that the functional and security requirements are being met, the lead developers must create and measure a form of metrics. Using metrics enables developers to quantify a characteristic of a given system and relate, compare, and visualize against a set standard (Reid 2022). By using and measuring metrics the software development team can measure and gauge if the program needs any modification based on the gathered metrics. [Metrics].

[SRS ?possibly if needed or relevant?]

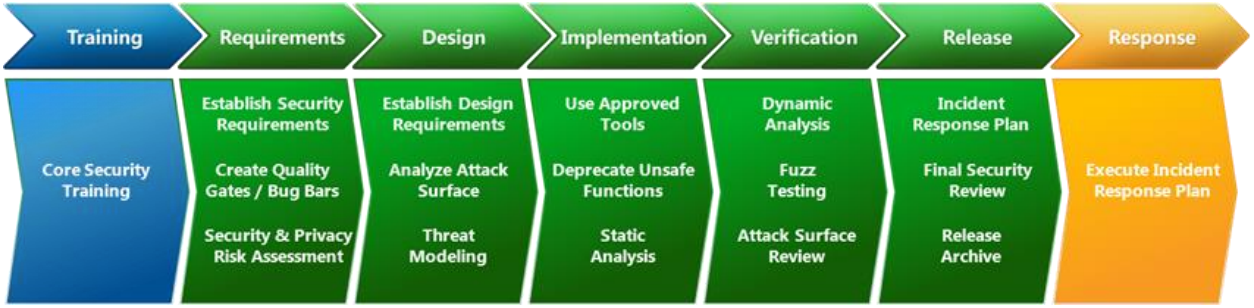
Secure Software Development Lifecycle

Even when a software development lifecycle is proposed and followed, there can be gaps and flaws in the security and functionality of the program. To mitigate these gaps, a Secure Software Development Lifecycle (SSDLC) should be chosen and followed in addition to the SDLC. By completing the activities and models in the different phases of the SSDLC, the benefits of both systems are used in conjunction with each other to ensure that the program is developed in a secure manner. There are many different types of SSDLCs, but almost all contain activities such as risk assessment, static analysis, dynamic analysis, response planning, and security review.

One of—if not the most used SSDLCs—is the Microsoft Security Development Lifecycle (MSDL). Shown in Figure 2, the MSDL contains seven phases each with three main activities that ensure the program is secure during and after development. The most important of these activities include, risk assessment, threat modeling, static testing, and dynamic testing. As such, during development of the Exxon sensors VT235x-c and PSX25, the development team will follow an SDLC and the MSDL. By the end of the development lifecycle, the program will have gone through both systems and as such reduces the likelihood of common vulnerabilities.

Figure 2

Microsoft Security Development Lifecycle



Note. This figure shows all the different phases of the Microsoft Security Development Lifecycle, as well as associated practices done within each phase. From "Microsoft Security Development Lifecycle", by Dansimp et al, (2022, November 8). (https://learn.microsoft.com/en-us/windows/security/threat-protection/msft-security-dev-lifecycle)

By utilizing this model in addition to the standard SDLC the development team can utilize a strategy called "shifting to the left." Jeff Levine (2022) describes shifting to the left as the process where a development team will preform common security practices earlier in the development process. Catching common flaws by preforming security practices early will dramatically cut costs in the development process, as when those flaws are left until later in the lifecycle, the development team will have to do steps over again.

Responsible, Accountable, Consulted, and Informed Chart

Another important step before development begins on the VT235x-c and PSX25 sensors, is that a Responsible, Accountable, Consulted, and Informed (RACI) Chart has to be made. A RACI chart provides clear and concise roles and responsibilities for each stakeholder and person involved in the development process. By utilizing a RACI chart, the development team will be able to know who is responsible for what tasks, as well as who should be consulted and informed

of any progress regarding the tasks. This also ensures that development team leaders and stakeholders are informed of and consulted—depending on the task at hand.

Table 1

Task:	Board of Directors	Dev. Team Lead	Developers	Designers	Engineer Team	Public Relations
Make Coding Standards	I	A	R	R	I	I
Facilitate SDLC	I	A	R	I	I	I
Manage Policies	I	R	A	C	I	I
Check Metric Thresholds	I	R	A	I	I	I
Incident Response Plan	C	I	I	R	I	A
Threat Modeling	I	A	R	I	I	I

Note. This table shows an example of a RACI chart that is used during a secure development process. In each box there can be an (R) for responsible, (A) for accountable, (C) for consulted, and (I) for informed.

By completing a RACI chart for various tasks—as shown in Table 1—not only does it show what group is responsible or accountable, but also who is informed about the decisions made about the task and who needs to be consulted about how a task is performed. By ensuring that each task in each phase of the MSDL is completed and consulted by the people who need to, it reduces the risk that any common vulnerability or flaw will make it to release.

Threat Modeling

[What is threat modeling? Why?]

Threat modeling is the process of creating a model to simulate common attacks on a program to catch common vulnerabilities and risks.

[Why does catching vulnerabilities early save money?]

[Shifting to the left]

[yadda yadda]

[What is DREAD? Why?]

[DREAD chart]

[What is STRIDE? Why?]

[STRIDE chart]

[2. Security Policies]

[How do we make sure it works?]

[Key policies that increase the probability that the developers are secure]

[**6 POLICIES**]

[1. MFA, 2. Least Priv]

[https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8778887/] USE PAPER ???

Security Policies

[3. Metrics]

[5 key metrics / threshold / what phase]

[1. Attack surface 2.] *Go back through my discussion post*

[Explain why it is critical / Who is it relevant to?]

[dashboard]

Metrics

[4. 3rd Party Aid]

[a. IBM AppScan]

[b. SAST Tool]

[5. Code Review for the PX25]

[a. How is the code secured from attacks]

[b. Propose a UI]

[Cover things in the OWASP top 10. Error handling etc]

[6. Audit]

[Check policies and procedures]

[7. Conclusion]

[a. Why my Security Plan is good]

[8. References]

References

Dansimp et al. (2022, November 8). *Microsoft Security Development Lifecycle*. Microsoft. https://learn.microsoft.com/en-us/windows/security/threat-protection/msft-security-dev-lifecycle

Levine, Jeff. (2022, July 8). *Invest early, save later: Why shifting security left helps your bottom line*. Google. https://cloud.google.com/blog/products/identity-security/shift-left-on-google-cloud-security-invest-now-save-later

OWASP. (2022). *OWASP in SDLC*. Owasp. https://owasp.org/www-project-integration-standards/writeups/owasp_in_sdlc/

Reid, Phillip. (2022). *App Sec Metrics* [PowerPoint slides]. Department of Cyber Security and Intelligence, Embry-Riddle Aeronautical University. https://erau.instructure.com/courses/146470/files/31563876/download?download_frd=1